

BrainChip AKD1000

Bring-up, model execution and troubleshooting manual



Álvaro Poveda Ruiz

User manual

Universidad Politécnica de Madrid, 2026

Version 1.2.3 · Revision: 3 October 2026

Abstract

This manual describes the bring-up of a BrainChip AKD1000 board connected over PCIe. The procedure covers host preparation, driver and Python environment installation, model loading and execution checks. It also explains how to measure latency and identify common faults.

The first test uses a simple model with a known output. It runs in software first and then on the board. This checks the Akida environment, model compatibility and hardware access separately before running your own application.

Physical execution of the complete revised procedure remains pending.

Scope

The procedure covers environment installation, board detection, model loading and execution, statistics and performance measurement. It also includes checks for connection, driver, data and model errors.

Apply the assembly instructions to the specific revision of each component. Before connecting power, check the voltage, polarity and cable orientation in the manufacturer's documentation. Board repair and internal electronic design are outside the scope of this manual.

How to use this document

Clone the repository or download and extract the source ZIP from GitHub. Open a terminal in `akd1000-bringup`, which contains `pyproject.toml`, `examples/` and `models/`. All project commands start from this directory unless another location is specified. Do not run the examples from the ZIP viewer. Use the PDF for the same version as the code.

Run only the block for your host. Blocks that use `sudo`, `apt` or `lspci` are for Linux. Chapter 4 includes environment activation on Windows.

Without a board, start at chapter 4 and complete the software test. With hardware available, first read the inventory and assembly steps in chapters 1 and 2, then prepare the driver and SDK.

Version and included material

Field	Value
Version and date	1.2.3; 3 October 2026
Editable source	<code>manual/en/main.tex</code> and <code>manual/en/sections/</code>
Tests for this edition	Automated tests and simulator runs with Akida 2.19.1 and 2.19.3; physical execution pending
Included model	<code>models/akida_v1_smoke_test.fbz</code> ; four inputs and two outputs

The bibliography links to the official instructions. The current software route and driver compatibility are checked separately: installing an SDK does not guarantee that the kernel can load the PCIe driver.

Contents

Abstract	2
How to use this document	3
1 Overview and workflow selection	7
1.1 System components	7
1.2 Three decisions before you start	7
1.3 Workflow matrix	8
1.4 Check each stage	8
2 Hardware, power and assembly	9
2.1 Assembly inventory	9
2.2 Supported topologies	9
2.2.1 Linux host with a PCIe slot	9
2.2.2 Raspberry Pi 5 with an FFC-to-PCIe adapter	9
2.3 Assembly	10
2.4 Example Waveshare carrier	10
2.5 Checks before power-on	11
3 Host preparation and PCIe driver	12
3.1 Initial checks	12
3.2 Enable PCIe on Raspberry Pi 5	12
3.3 Check the PCIe connection	12
3.4 Install the driver	13
3.5 Check the driver	13
4 Software environment and first test	14
4.1 Prepare the environment	14
4.2 Install Akida	14
4.3 First software test	14
4.4 Check the host	15
4.5 First execution on the AKD1000	15
5 Running Akida models	16
5.1 Load and inspect a model	16
5.2 Select the device	16
5.3 Map the model to the board	16
5.4 Prepare the input	17
5.5 Run the model	17
5.6 Clock mode	18
6 Performance and energy measurement	19
6.1 What is being measured	19
6.2 Latency measurement	19
6.3 Run a benchmark	19
6.4 Power and energy	20
6.5 Save the results	20

7	Troubleshooting	21
7.1	Information for diagnosing a fault	21
8	Maintenance and updates	23
8.1	Save a known working configuration	23
8.2	Update the environment	23
8.3	Shutdown and storage	23
A	Quick command reference	24
A.1	Check the system	24
A.2	Raspberry Pi 5	24
A.3	PCIe driver	24
A.4	Python environment	24
A.5	Smoke tests	25
A.6	Run your own model	25
A.7	Benchmark	25
A.8	Quick troubleshooting	25
B	Sources and limitations	26

List of Figures

2.1	Raspberry Pi 5, carrier and AKD1000 assembly	10
2.2	Powered assembly	11
6.1	Latency measurement boundary	19

List of Tables

1.1	Role of each system layer.	7
1.2	Workflows covered by the manual.	8
2.1	Basic assembly information.	9
7.1	Quick troubleshooting guide.	21

1 Overview and workflow selection

1.1 System components

The AKD1000 does not operate as a standalone application. The host runs the operating system and runtime; PCIe carries commands and data; the driver exposes the device to the system; and the FBZ model defines the network that the runtime can map. Separating these layers avoids looking for a power fault in Python or reinstalling the driver to fix a missing package.

Table 1.1: Role of each system layer.

Layer	Role	What it does not prove on its own
Host	Runs Linux or Windows, Python, data preparation and logging.	That the computation ran on the board.
PCIe interface	Carries configuration, inputs and outputs.	That a usable driver is available.
Linux driver	Provides device access, usually through <code>/dev/akida*</code> .	That the model is compatible.
Akida runtime	Loads FBZ files, discovers devices, maps and executes models.	That virtual mapping is physical inference.
AKD1000	Runs the sequences assigned to the hardware.	Total Python execution time or total platform power consumption.
FBZ	Stores a serialized Akida network.	The meaning of inputs, labels or the dataset.

The PCIe board documented by BrainChip contains an AKD1000, uses a PCIe host interface and has LPDDR4 memory accessible through DMA [2]. The SoC product brief describes twenty neural processing cores and a two-lane PCIe 2.1 endpoint [1]. These are product specifications, not a guarantee of latency, energy or resource utilization for every network.

1.2 Three decisions before you start

Decision 1: software or hardware. Without a board, follow route **S**: software environment, test model and virtual device. With a board and a documented PCIe connection, complete route **S** first, then continue with route **H**. Simulation checks the model and package; it does not check the cabling, driver or board.

Decision 2: current or reference environment. To learn with the current documentation, use route **A**: MetaTF 2.19.3, or the version listed on the official page when you read this manual. To repeat the included test, use route **R**: Python 3.11 and Akida 2.19.1. Do not combine both installations in one virtual environment.

Decision 3: host type. A Linux PC with a PCIe slot and a Raspberry Pi 5 with an FFC adapter need different assemblies. Windows is included in the current MetaTF software configurations [5], but this manual does not claim physical board support on Windows: the public PCIe driver instructions consulted are for Linux [3].

1.3 Workflow matrix

Table 1.2: Workflows covered by the manual.

Route	Host	Objective	Final evidence
S	Compatible Windows or Linux host, no board	Install MetaTF, import Akida and run in software.	Recorded version, model output and virtual mapping.
H-PC	Linux host, board and documented PCIe connection	Detect the AKD1000 and run on it through the public driver.	<code>lspci</code> , module, node, <code>akida devices</code> and physical test.
H-Pi	Raspberry Pi 5, FFC adapter and board	Enable PCIe when needed and complete H-PC.	Boot configuration, enumeration and physical test.
R	Python 3.11 and reference SDK	Repeat the smoke test with Akida 2.19.1.	Known output and verified virtual mapping.

1.4 Check each stage

Each stage answers a different question:

1. **Inventory:** do I know the exact host, board, adapter, power supplies, operating system, kernel and architecture?
2. **Assembly:** is the chain connected according to the documentation for those revisions?
3. **Enumeration:** does the host see a new PCIe endpoint?
4. **Driver:** has the module been built for this kernel, and is the node accessible?
5. **SDK:** does the active interpreter import the expected Akida version?
6. **Model:** does the FBZ load, have the expected contract and map as intended?
7. **Execution:** does a known input produce a known output on hardware?
8. **Measurement:** are the window, units and power measurement boundary stated?

Resolve the first failed stage before continuing. Save its error to help locate the fault.

2 Hardware, power and assembly

2.1 Assembly inventory

Before starting, record the information you can check for each component. Do not infer a part number from a photograph or its similarity to another model. If a detail such as a board revision or supply voltage cannot be confirmed from the label, invoice or manufacturer's documentation, mark it as *unverified*.

Table 2.1: Basic assembly information.

Component	Information	Source
Host	Manufacturer, model, RAM and revision	Host, firmware or invoice
Operating system	Distribution, version and architecture	<code>/etc/os-release</code> , <code>uname -m</code>
Kernel	Full version	<code>uname -r</code>
Akida board	Model and revision	Label and BrainChip documentation
Adapter	Manufacturer, model and link type	Board markings and manufacturer's documentation
Cable	Type and orientation	Markings and adapter documentation
Host power supply	Voltage and power	Power supply label
Board or carrier power supply	Voltage, polarity and connector	Label and manufacturer's documentation

M.2, Mini PCIe, conventional PCIe and the Raspberry Pi 5 FFC connector are not equivalent. Always check the connection type and cable orientation in the documentation for the hardware you are using.

2.2 Supported topologies

2.2.1 Linux host with a PCIe slot

On a Linux PC, install the board in a compatible PCIe slot and power it as specified in its documentation. Shut down and disconnect the host before inserting or removing the board.

After assembly, continue with chapter 3. If `lspci` does not detect a new PCIe device, resolve that problem before configuring Python.

2.2.2 Raspberry Pi 5 with an FFC-to-PCIe adapter

Raspberry Pi 5 has a PCIe Gen 2 $\times 1$ interface accessible through an FFC connector [6]. A conventional PCIe board needs an adapter that routes this link to a compatible slot.

Even if the adapter has a physically longer slot, the Raspberry Pi 5 link remains $\times 1$.

Figure 2.1 shows an example assembly.

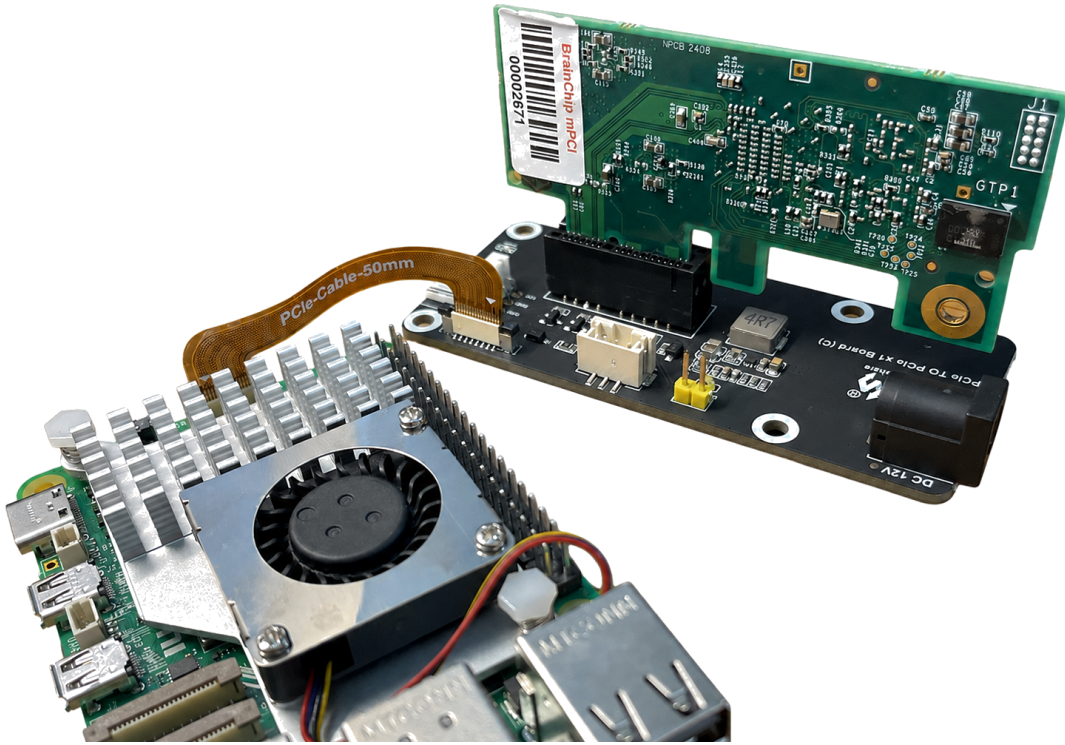


Figure 2.1: Assembly consisting of a Raspberry Pi 5, a PCIe adapter and the BrainChip AKD1000 board in the example assembly.

2.3 Assembly

If the manufacturer specifies a different connection sequence, follow that procedure.

1. Shut down the host and disconnect the power supplies.
2. Check the FFC cable orientation and the adapter or board supply voltage.
3. Connect the FFC cable to the adapter in the specified orientation.
4. Insert the AKD1000 fully into the carrier's PCIe slot.
5. Inspect the connections before powering the system again.

Electrical fault. Disconnect power if you notice smoke, an unusual smell, excessive temperature or any other sign of a possible power or assembly fault.

2.4 Example Waveshare carrier

The example assembly shows a Waveshare adapter connected to the Raspberry Pi 5 through a 50 mm FFC cable. The board is marked DC 12V and *PCIe TO PCIe x1 Board (C)*.

Waveshare's current documentation describes this model as having a slot mechanically compatible with PCIe $\times 1$, $\times 4$, $\times 8$ and $\times 16$ boards [7]. The Raspberry Pi 5 link remains PCIe $\times 1$ in every case.

The photographs are illustrative and have edited backgrounds. They do not establish the adapter item number or revision on their own. Check these details on the actual component and its documentation before connecting it.



Figure 2.2: Example powered assembly.

2.5 Checks before power-on

Before powering the assembly, check that the FFC cable has the correct orientation, the board is fully seated and the supply voltages match those specified for the components.

Before power-on. If you cannot confirm the voltage, polarity or orientation of a connection, do not power the assembly until you have checked the relevant documentation.

3 Host preparation and PCIe driver

3.1 Initial checks

Before installing the driver, check the operating system version, architecture and kernel. On Linux, you can save this information in `evidence`:

```
mkdir -p evidence
{
  cat /etc/os-release
  uname -r
  uname -m
  lspci -nn
} | tee evidence/host-inventory.txt
```

`uname -m` usually reports `x86_64` on a PC and `aarch64` on a Raspberry Pi 5 with a 64-bit operating system. The kernel version is needed to check driver compatibility.

3.2 Enable PCIe on Raspberry Pi 5

This section applies only to Raspberry Pi 5. Compatible HAT+ devices can be detected automatically, whereas Raspberry Pi specifies that a non-HAT+ PCIe adapter needs the PCIe interface enabled in the configuration file [6].

On current Raspberry Pi OS, open:

```
sudoedit /boot/firmware/config.txt
```

If that path does not exist, check `/boot/config.txt`. Add:

```
dtparam=pciex1
```

Save the file and reboot:

```
sudo reboot
```

Raspberry Pi 5 uses PCIe Gen2 by default. Gen3 can be enabled, but Raspberry Pi warns that the board is not certified for stable operation at that speed [6]. Keep Gen2 for bring-up.

3.3 Check the PCIe connection

After booting, check that the system detects the board:

```
lspci -nn
```

A PCIe device corresponding to the board should appear. If it does not, check the assembly, power and, on Raspberry Pi 5, whether PCIe is enabled before proceeding with the driver.

For more information about the detected device, run:

```
lspci -nnk
```

This stage checks only the PCIe connection. Seeing the board in `lspci` does not yet mean that the Akida driver is installed.

3.4 Install the driver

BrainChip's official PCIe driver is distributed in the `akida_dw_edma` repository. Before installing it, check the kernel version:

```
uname -r
```

The driver version consulted for this manual supports kernels from 5.4 to 6.8. The module does not build on kernels 6.9 or later [3]. Compatibility can change, so also check the repository README before installation.

Unsupported kernel. If the kernel version is outside the driver's supported range, do not proceed with installation until you have checked whether BrainChip has released a compatible version.

On Ubuntu, install the dependencies specified by BrainChip:

```
sudo apt update
sudo apt install dkms build-essential "linux-headers-$(uname -r)"
```

Package names may differ on other distributions, but DKMS, build tools and headers for the running kernel must be installed.

Then clone the repository and install the driver:

```
git clone https://github.com/Brainchip-Inc/akida_dw_edma.git
cd akida_dw_edma
git rev-parse HEAD
./install.sh
```

The output of `git rev-parse HEAD` records the exact driver revision used; you can also use your own configuration.

3.5 Check the driver

After installation, check the DKMS status and confirm that the Akida device is available:

```
dkms status
lsmod | grep -i akida
ls -l /dev/akida*
lspci -nnk
```

Installation is successful when the device still appears on PCIe, the module is loaded and at least one `/dev/akida*` node exists.

If the device appears in `lspci` but `/dev/akida*` is missing, read the kernel messages:

```
sudo dmesg | grep -iE 'akida|edma|pcie' | tail -n 120
```

If the node exists but is accessible only as `root`, check its permissions and the udev rule installed by the driver before proceeding.

With the driver working, continue with the software environment setup in chapter 4.

4 Software environment and first test

4.1 Prepare the environment

BrainChip distributes the Akida tools through MetaTF. For the version used in this manual, the current installation uses `metatf==2.19.3` and supports Python 3.10–3.12 [5].

The repository also includes a reference configuration with Python 3.11 and Akida 2.19.1 to repeat the smoke test in a known environment.

Open a terminal in `akd1000-bringup`, the directory containing the project source.

On Linux:

```
python3.11 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
```

On Windows, in PowerShell:

```
py -3.11 -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
```

Create the virtual environment only once. In a new terminal, activate it again before proceeding.

4.2 Install Akida

Choose one of the following two configurations.

For the current version documented by BrainChip:

```
python -m pip install "metatf==2.19.3"
```

To repeat the reference configuration included with the repository:

```
python -m pip install -r requirements/reproducible.txt
```

Do not combine both installations in one virtual environment.

Then install the tools included in this repository:

```
python -m pip install -e . --no-deps
python -m pip check
```

Check that Akida imports successfully:

```
python -c "import akida; print(akida.__version__); print(akida.devices())"
```

Without a connected board, `akida.devices()` returning `[]` is normal. With an AKD1000 connected and a working driver, at least one device should appear.

4.3 First software test

Before using the board, run the test model included with the repository:

```
python examples/00_simulator_smoke.py
```

The model receives four input values. For

```
[3, 2, 1, 4]
```

the expected output is

```
[5, 5]
```

The script first runs the model in software, then checks that it can map to a virtual AKD1000. This last step verifies model compatibility with the board's architecture; it does not use physical hardware.

You can also run the check provided by the repository's command-line tool:

```
akd1000-bringup verify --id smoke --model models/akida_v1_smoke_test.fbz
```

If both tests pass, the Python environment and test model are ready.

4.4 Check the host

Before using the physical board, you can run the repository's diagnostics:

```
akd1000-bringup doctor
```

To save the result:

```
mkdir -p evidence
akd1000-bringup doctor --json > evidence/preflight.json
```

The tool checks the Akida version, kernel, PCIe connection and detected devices, among other information. Its output helps locate the fault when the board does not appear.

On a host without an AKD1000, a diagnostic report showing no detected devices is normal.

4.5 First execution on the AKD1000

This test requires PCIe detection of the board and the driver from chapter 3 to be installed.

First check that Akida detects the device:

```
akida devices
```

Then run:

```
python examples/01_hardware_smoke.py
```

The script loads the same model used in the software test, maps it to the AKD1000 and runs the known input. The output should again be:

```
[5, 5]
```

A PASS result confirms that the model has loaded, the board is accessible through Akida and inference has run successfully on the hardware.

If the physical test fails. Do not move on to your own model yet. Save the error message and use the procedure in chapter 7 to locate the fault.

5 Running Akida models

5.1 Load and inspect a model

Akida models are distributed as FBZ files. Before running one, check at least its input shape, output shape and the preprocessing used during conversion.

Load the model and inspect its structure:

```
import akida

model = akida.Model("models/model.fbz")

print("input_shape:", model.input_shape)
print("output_shape:", model.output_shape)
print("ip_version:", model.ip_version)
model.summary()
```

`summary()` shows the layers and how the model can run. Resource use depends on the model and mapping mode.

5.2 Select the device

`akida.devices()` returns the available physical devices [4]:

```
import akida

devices = list(akida.devices())

if not devices:
    raise RuntimeError("No Akida device detected")

device = devices[0]
print(device)
```

A device created with `akida.AKD1000()` is virtual and checks model mapping. To run inference on the board, use a device returned by `akida.devices()`.

5.3 Map the model to the board

A loaded model can initially run in software. To use the AKD1000, map it to the device:

```
model.map(
    device,
    hw_only=True,
    mode=akida.MapMode.AllNps
)

model.summary()
```

`hw_only=True` requires execution entirely on Akida hardware [4]. If the model cannot map this way, the call raises an error.

`AllNps`, `HwPr` and `Minimal` allocate AKD1000 resources in different ways. For a first run, you can use `AllNps`. Chapter 6 examines its effect on latency and power consumption.

5.4 Prepare the input

Provide the input as a NumPy uint8 tensor. Its shape, effective range and channel order must match those used during model conversion. Check each sample shape against `model.input_shape`. For example, for data stored in NumPy:

```
import numpy as np

data = np.load("data/inputs.npy", allow_pickle=False)

print("shape:", data.shape)
print("dtype:", data.dtype)
print("min/max:", data.min(), data.max())

if data.dtype != np.uint8:
    raise TypeError("Akida model inputs must use uint8")

if tuple(data.shape[1:]) != tuple(model.input_shape):
    raise ValueError(
        f"Input shape {data.shape[1:]} "
        f"!= {model.input_shape}"
    )
```

The effective range depends on the model input precision. The tool rejects other data types without automatically converting them to uint8.

5.5 Run the model

The Akida API can run the model through `forward()` or `predict()`. The choice depends on how the model output was defined and validated during conversion [4].

For example:

```
output = model.forward(data)
print(output)
```

Keep the same execution method throughout a measurement campaign so that the results are comparable.

You can also use the tool included in this repository:

```
akd1000-bringup run \  
  --model models/model.fbz \  
  --input data/input.npy \  
  --method forward \  
  --clock Performance \  
  --map AllNps \  
  --output outputs/raw-output.npy
```

The tool loads the input, runs the model and saves the output to a NumPy file. The input must have the shape

```
(batch, *model.input_shape)
```

Models with multiple inputs or outputs require direct handling through the Akida API.

5.6 Clock mode

The AKD1000 has Performance, Economy and LowPower clock modes. Select the mode through the API:

```
device.soc.clock_mode = akida.soc.ClockMode.Performance  
print(device.soc.clock_mode)
```

Record the clock mode when comparing performance measurements. Lower instantaneous power does not necessarily mean lower energy per inference, because execution time may also change.

6 Performance and energy measurement

6.1 What is being measured

Before comparing results, keep the model, data and execution configuration fixed. Also record the clock mode, mapping mode and number of repetitions.

6.2 Latency measurement

The tool measures the time taken by the model execution call. This includes communication between the host and board, the SDK call and inference on the AKD1000.

File loading, model mapping and warm-up runs occur before measurement starts.

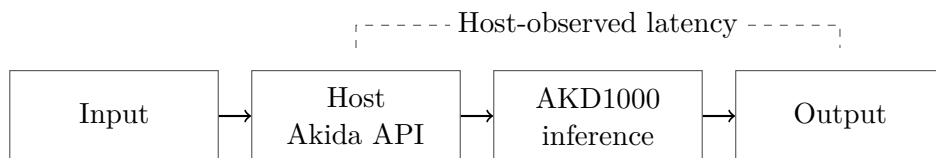


Figure 6.1: Elements included in the latency measurement from the host.

Results in this manual are therefore expressed as *host-observed latency*. They do not come from an isolated internal AKD1000 timer.

To measure the full execution time of an application, also include acquisition, preprocessing and postprocessing.

6.3 Run a benchmark

Before a full campaign, run a short test to check the model, data and configuration.

```
akd1000-bringup benchmark \  
  --model models/model.fbz \  
  --input data/inputs.npy \  
  --method forward \  
  --clock Performance \  
  --map AllNps \  
  --warmup 8 \  
  --repetitions 1 \  
  --output outputs/smoke
```

The output directory contains:

- `latencies.csv`, with individual measurements;
- `summary.json`, with the configuration and trial summary;
- `host.json`, with information about the host used.

Check that the sample count, units and configuration are as expected. If the test passes, repeat the benchmark with the required number of repetitions and save the results in a different directory.

To compare configurations, change one variable at a time. For example, keep the model fixed and compare `AllNps`, `HwPr` and `Minimal`, or keep the mapping fixed and change the clock mode.

6.4 Power and energy

Akida can enable device power measurement and report statistics after inference [4]:

```
device.soc.power_measurement_enabled = True

model.forward(data)

print(model.statistics)
```

The power reported by this API is the measurement available on the device. It does not represent the total consumption of the Raspberry Pi, carrier and board.

Given average power and host-observed latency, estimated energy per inference can be calculated as

$$E_{\text{est}} [\mu\text{J}] = P_{\text{API}} [\text{mW}] t_{\text{host}} [\text{ms}]. \quad (6.1)$$

The unit relationship is direct:

$$1 \text{ mW} \cdot 1 \text{ ms} = 1 \mu\text{J}.$$

This estimate combines API power with host-observed latency. It helps compare runs made with the same procedure; it measures neither internal chip energy nor total system energy consumption.

Measuring total consumption would require external instrumentation on the assembly's power supply.

6.5 Save the results

For each trial, retain at least:

- the model and its version;
- the input data;
- the clock mode;
- the mapping mode;
- the number of repetitions;
- the files generated by the benchmark.

When comparing platforms such as CPU, GPU or FPGA, use the same task, data and an equivalent measurement definition.

7 Troubleshooting

When a fault occurs, check the system in the same order as bring-up: PCIe connection, driver, Python environment, model and data. Table 7.1 summarizes common faults and where to start looking.

Table 7.1: Quick troubleshooting guide.

Symptom	Check	Action
The board does not appear in <code>lspci</code> .	Power, FFC cable, assembly and Raspberry Pi 5 PCIe configuration.	Check the physical connection and PCIe configuration first.
The board appears on PCIe, but <code>/dev/akida*</code> does not exist.	Kernel version, headers, <code>dkms status</code> , <code>lsmod</code> and <code>dmesg</code> messages.	Check driver installation and compatibility.
<code>/dev/akida*</code> exists, but <code>akida.devices()</code> is empty.	Active virtual environment, Akida version and device permissions.	Check that you are using the right environment and that the user can access the device.
<code>import akida</code> fails.	Python version and packages installed in the active environment.	Activate the right virtual environment or reinstall Akida in it.
The FBZ file does not load.	File path, Akida version and model compatibility.	First check the test model included with the repository.
The model cannot map with <code>hw_only=True</code> .	<code>model.summary()</code> output, layers used and model version.	Check that the model supports execution entirely on the AKD1000.
The input is rejected.	Data shape, type, range and order.	Compare the input with <code>model.input_shape</code> and the pre-processing used during conversion.
Execution finishes, but the output is not as expected.	Preprocessing, sample order and use of <code>forward()</code> or <code>predict()</code> .	First test an input with a known expected result.
Latency varies widely between runs.	Clock mode, mapping mode, warm-up, host load and number of repetitions.	Repeat the measurement with a fixed configuration.
No power measurement appears.	Power measurement enabled and statistics returned by Akida.	Check the configuration and increase the test duration if needed.

7.1 Information for diagnosing a fault

The repository’s tool can collect much of the information needed to locate a fault:

```
mkdir -p evidence
akd1000-bringup doctor --json > evidence/doctor.json
```

For more detail, also save the system and driver status:

```
uname -a > evidence/uname.txt
lspci -nnk > evidence/lspci-nnk.txt
dkms status > evidence/dkms.txt
ls -l /dev/akida* > evidence/device-nodes.txt 2>&1
python -m pip freeze > evidence/pip-freeze.txt
sudo dmesg | grep -iE 'akida|edma|pcie' > evidence/driver-log.txt
```

When reporting a fault, also retain the command that caused it and the full error message. This usually helps distinguish a PCIe, driver, Python environment, model or data problem.

8 Maintenance and updates

8.1 Save a known working configuration

After the first successful run on the board, save the basic environment information. It provides a reference if an update causes a fault.

```
mkdir -p evidence/known-good

akd1000-bringup doctor --json \
  > evidence/known-good/doctor.json

python -m pip freeze \
  > evidence/known-good/pip-freeze.txt

sha256sum models/*.fbz \
  > evidence/known-good/model-hashes.txt
```

Also retain the project and driver versions used. If the system later stops working, this configuration helps identify what has changed.

8.2 Update the environment

Updates to the kernel, driver, Python or Akida can affect board operation. Where possible, change one component at a time and repeat the smoke test afterwards.

A simple procedure is:

1. check compatibility with the new version;
2. apply the update;
3. reboot if needed;
4. repeat board detection and the software and hardware tests.

If a fault occurs, compare the system with the saved configuration and follow the procedure in chapter 7.

The driver uses DKMS to rebuild the module after a kernel update. This works only while the new kernel version remains supported by the driver.

8.3 Shutdown and storage

Before disconnecting the board, finish active runs and shut down the host normally. Then disconnect the power supplies before removing the board or FFC cable.

For long-term storage, protect the boards against electrostatic discharge and avoid bending the FFC cable or leaving it under mechanical tension.

A Quick command reference

This appendix collects the main Linux commands in the procedure. The corresponding chapters explain their use and conditions.

A.1 Check the system

```
uname -r
uname -m
cat /etc/os-release
lspci -nn
```

A.2 Raspberry Pi 5

For a non-HAT+ PCIe adapter, edit:

```
sudoedit /boot/firmware/config.txt
```

and add this line if it is not already present:

```
dtparam=pciex1
```

Then reboot and check the connection:

```
sudo reboot
lspci -nn
```

A.3 PCIe driver

On Ubuntu:

```
sudo apt update
sudo apt install dkms build-essential "linux-headers-$(uname -r)"
```

Download and install the driver:

```
git clone https://github.com/Brainchip-Inc/akida_dw_edma.git
cd akida_dw_edma
git rev-parse HEAD
./install.sh
```

Check the installation:

```
dkms status
lsmod | grep -i akida
ls -l /dev/akida*
lspci -nnk
```

A.4 Python environment

From the project root:

```
python3.11 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
```

Current MetaTF installation:

```
python -m pip install "metatf==2.19.3"
```

or the repository's reference environment:

```
python -m pip install -r requirements/reproducible.txt
```

Then install the project tools:

```
python -m pip install -e . --no-deps
python -m pip check
```

A.5 Smoke tests

Software test:

```
python examples/00_simulator_smoke.py
akd1000-bringup verify \
  --id smoke \
  --model models/akida_v1_smoke_test.fbz
```

Diagnostics:

```
akd1000-bringup doctor
```

Board test:

```
akida devices
python examples/01_hardware_smoke.py
```

A.6 Run your own model

```
akd1000-bringup run \
  --model models/model.fbz \
  --input data/input.npy \
  --method forward \
  --clock Performance \
  --map AllNps \
  --output outputs/raw-output.npy
```

A.7 Benchmark

```
akd1000-bringup benchmark \
  --model models/model.fbz \
  --input data/inputs.npy \
  --method forward \
  --clock Performance \
  --map AllNps \
  --warmup 8 \
  --repetitions 1 \
  --output outputs/smoke
```

A.8 Quick troubleshooting

1. If the board does not appear in `lspci`, check the assembly and PCIe configuration.
2. If it appears on PCIe but `/dev/akida*` is missing, check the driver.
3. If the node exists but `akida.devices()` is empty, check the environment and permissions.
4. If the test model works but your own model does not, check the model and its input data.

B Sources and limitations

The instructions are based on official BrainChip, Raspberry Pi and WaveShare documentation. The repository's reference environment uses Akida 2.19.1; the documented MetaTF route uses 2.19.3. Check driver compatibility against your kernel and SDK. A complete session with a new user and a physical board remains pending.

API power readings do not measure total system consumption. The manual does not cover *edge learning* or Akida Engine.

The AoA model-conversion pipeline and its experimental results are documented separately in `akida-model-deployment-pipeline`.

Bibliography

- [1] BrainChip, Inc. *AKD1000 Akida System-on-Chip Product Brief*, August 2025. Version 2.3, August 2025. <https://brainchip.com/wp-content/uploads/2025/08/Akida-AKD1000-SoC-Product-Brief-V2.3-Aug.25.pdf>.
- [2] BrainChip, Inc. *AKD1000 PCIe Card Hardware Product Brief*, 2025. Product Brief v1.0. <https://brainchip.com/wp-content/uploads/2025/04/AKD1000-PCIe-Card-Product-Brochure-v1.0.pdf>.
- [3] BrainChip, Inc. Akida pcie driver: akida_dw_edma. Code repository, 2026. Master branch accessed on 29 September 2026. https://github.com/Brainchip-Inc/akida_dw_edma.
- [4] BrainChip, Inc. *Akida User Guide*, 2026. MetaTF 2.19.3; Accessed on 29 September 2026. https://doc.brainchipinc.com/user_guide/akida.html.
- [5] BrainChip, Inc. *MetaTF Documentation: Installation*, 2026. Accessed on 29 September 2026. <https://doc.brainchipinc.com/installation.html>.
- [6] Raspberry Pi Ltd. *Raspberry Pi Computer Hardware: PCIe Connector*, 2026. Accessed on 29 September 2026. <https://www.raspberrypi.com/documentation/computers/raspberry-pi.html#raspberry-pi-connector-for-pcie>.
- [7] Waveshare. *PCIe TO PCIe x1 Board (C)*, 2026. Connection guide accessed on 29 September 2026. [https://www.waveshare.com/wiki/PCIe_TO_PCIe_x1_Board_\(C\)](https://www.waveshare.com/wiki/PCIe_TO_PCIe_x1_Board_(C)).